Bioinformatics
doi.10.1093/bioinformatics/xxxxxx
Advance Access Publication Date: Day Month Year
Original Paper

OXFORD

Sequence analysis

dinara-align: hardware-aware algorithms for exact pairwise alignment

Konstantinos Kyriakidis [1,*], **Benedict Paten** [1,*]

[1]UC Santa Cruz Genomics Institute, Santa Cruz, CA, USA

*To whom correspondence should be addressed: kkyriaki@ucsc.edu, bpaten@ucsc.edu

Associate Editor: XXXXXXX

Received on XXXXX; revised on XXXXX; accepted on XXXXX

Abstract

Motivation: Exact pairwise alignment returns an optimal alignment that no heuristic cut-off can change, the reference for evaluating read mappers, assemblers and variant callers. Yet no exact algorithm is fastest across workloads: diagonal transition wins on near-identical pairs, bit-parallel band doubling on long divergent reads, and inter-sequence vectorisation on batches of short pairs.

Results: We present dinara-align, an exact aligner that chooses its algorithm for each pair. Every hand-over that rests on a projection provably bounds its waste; a gap-affine alignment adds a second search only when its measured work exceeds what two would need, provably still finding an optimal alignment; short pairs are aligned in bands that each vector lane certifies optimal; and, within the memory limit, all engines break ties alike at unit and gap-affine costs. On one Skylake-X core, dinara-align was the fastest exact aligner compared on every dataset at unit costs: up to 2.4 times faster than A*PA2 on ultra-long nanopore reads, and 1.95 and 5.2 times faster than WFA on Illumina and HiFi reads. At gap-affine costs it was the fastest on every dataset any tool completed.

Availability and implementation: dinara-align is free software under the Mozilla Public License 2.0, available at `https://github.com/kokyriakidis/dinara-align` with C, C++, Python and command-line interfaces.

1 Introduction

Pairwise alignment finds the least costly series of substitutions, insertions and deletions that transforms one sequence into another. It underlies read mapping, variant detection, genome assembly and multiple sequence alignment (Prjibelski *et al.*, 2019): read mappers align each read to the reference window its seeds select, assemblers compare reads with one another to find their overlaps (Li, 2018), and assemblies, haplotypes and viral genomes are compared base by base.

Read lengths have grown from hundreds of bases to hundreds of kilobases. The gap-affine dynamic program (Gotoh, 1982) fills an $n \times m$ matrix, 2.5×10^{11} cells for one pair of 500 kbp reads, so heuristic aligners compute only part of it: a fixed or adaptive band (Liu and Steinegger, 2023), a series of windows, or the cells before the score drops by more than a threshold (X-drop, or Z-drop in minimap2 (Li, 2018)). Their result then depends on these cut-offs, and alignments with large indels or clustered errors can be missed or misplaced. Exact aligners instead skip only cells that provably cannot lie on an optimal path, so that their running time grows with the distance as well as the length, rather than with the product of the lengths.

Three ideas underlie current exact aligners. *Band doubling* computes only the diagonals a path within a cost bound can reach, doubling the bound until the optimum lies within it (Ukkonen, 1985); *bit-parallelism* encodes the score differences of 64 cells of a column in two machine words (Myers, 1999), and Edlib combines the two (Šošić and Šikić, 2017). *Diagonal transition* extends the furthest-reaching paths one cost at a time (Ukkonen, 1985; Myers, 1986); WFA extended it to gap-affine costs (Marco-Sola *et al.*, 2021) and BiWFA made its memory linear in the distance (Marco-Sola *et al.*, 2023). A*PA guided an A* search by a gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024); A*PA2 combined that heuristic with band doubling over SIMD blocks, aligning ultra-long reads up to 19 times faster than Edlib and BiWFA (Groot Koerkamp, 2024a); and QuickEd aligns within a bound it finds heuristically (Doblas *et al.*, 2025). Full-matrix vectorisation, in anti-diagonal (Wozniak, 1997), striped (Farrar, 2007; Zhao *et al.*, 2013), prefix-scan (Daily, 2016), difference-recurrence (Suzuki and Kasahara, 2018; Li, 2018) and inter-sequence (Rognes, 2011; Rahn *et al.*, 2018) layouts, takes time in proportion to the matrix or the band instead.

© The Author 2026. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

Each of these methods is fastest on part of the range of workloads: diagonal transition, whose expected work grows with the square of the distance, on short and near-identical pairs; bit-parallel band doubling, whose work grows with the length times the distance, on long divergent reads, with the seed heuristic on ultra-long ones; and inter-sequence vectorisation on batches of short pairs. An aligner that is fastest across workloads must therefore choose among them for each pair, and an exact aligner must make every such choice (which algorithm, which bound, when to abandon a search) without giving up optimality. In dinara-align every engine is exact, so no choice changes the optimal cost, and every hand-over that rests on a projection provably bounds the work it can waste. Our contributions are:

1. **One wavefront search or two, chosen by measured work** (Section 3.9): a gap-affine alignment adds a second search only when its measured work exceeds what two would need, and still returns an optimal alignment (Proposition 2).
2. **Projections with bounded waste** (Sections 3.3 and 3.10): a projected distance sets a band's first bound only when two projections agree, and a search hands over to an algorithm projected to be cheaper only after spending one twenty-fourth of that algorithm's least possible work.
3. **Certified bands for inter-sequence batches** (Section 3.6): each pair is aligned in a vector lane within a narrow band that the lane certifies optimal under gap-affine and two-piece costs (Proposition 1), bringing the speculate-and-verify approach of narrow-band aligners (Gibrat, 2018; Fujiki *et al.*, 2020) to every lane of a batch.
4. **Direct neighbour construction for the seed heuristic** (Section 3.5): inexact neighbours of exact seed matches are inserted without search, provably including every one that local pruning keeps (Lemma 1); we also prove that A*PA2's pruning keeps the heuristic a lower bound (Lemma S1).
5. **One alignment whichever engine computes it** (Sections 3.1 and 3.7): at unit, gap-affine and two-piece costs every engine breaks ties by the same rule, unless a gap-affine pair must be split to fit the memory limit, and a unit-cost band recomputes its traceback from recorded state rather than splitting the pair, so that its alignment is the same under any memory limit.
6. **Engines arranged for the processor** (Sections 3.2, 3.4 and 3.11): diagonal transition with gathered AVX-512 loads, interleaved bit-parallel groups with a shorter dependency chain, and lower per-pair overhead on short reads.
7. **One library** (Table 1; Supplementary Table S3) for global, ends-free, extension and local alignment under unit, gap-affine, two-piece and substitution-matrix costs, for single pairs and batches, through C, C++, Python and command-line interfaces.

We evaluate dinara-align on the datasets of the A*PA2 and QuickEd benchmarks, with QuickEd's Illumina and HiFi sets replaced by HG002 reads, each paired with the stretch of HG002's own assembly it covers.

2 Background

2.1 Alignment and costs

Let $A = a_1 \dots a_n$ be the reference and $B = b_1 \dots b_m$ the query, and let cell (i, j) lie on diagonal $j - i$. Under gap-affine costs (x, o, e), for example (4, 6, 2), a match costs 0, a mismatch x and a gap of k letters $g(k) = o + ke$; unit costs have $x = e = 1$ and $o = 0$, two-piece costs take the minimum of two such functions (Li, 2018), and a substitution matrix instead scores each pair of letters. Global alignment compares two whole sequences; ends-free alignment places a read inside a longer window or finds the overlap of two reads; extension grows an alignment from a seed match (Li, 2018); and local alignment finds the best-matching parts of two sequences. Extension and local alignment maximise a score that rewards each match (Section 3.8). An alignment is *optimal* if no alignment has lower cost under the requested costs and mode; a caller may further restrict alignments to a band of diagonals or a maximum cost, and an exact aligner then returns an optimal alignment among those or reports that none exists. At unit costs the distance $D(i, j)$ between the prefixes $a_1 \dots a_i$ and $b_1 \dots b_j$ satisfies

$$D(i,j) = \min\big(D(i-1,j)+1,\ D(i,j-1)+1,\ D(i-1,j-1)+[a_i \neq b_j]\big), \tag{1}$$

where $[a_i \neq b_j]$ is 1 if the letters differ and 0 otherwise, with $D(i, 0) = i$ and $D(0, j) = j$, and the distance is $D(n, m)$; gap-affine costs need three such layers (Gotoh, 1982).

2.2 Band doubling and bit-parallel computation

At unit costs, an alignment of cost at most t visits only diagonals d with $|d| + |d - \delta| \leq t$, where $\delta = m - n$; band doubling computes only those cells, doubling t until $D(n, m) \leq t$ (Ukkonen, 1985). Myers' recurrence advances the score differences of 64 cells of a column, held in two machine words, by one column at a time (Myers, 1999), and A*PA2 computes the band in tiles of 256 columns with SIMD vectors (Groot Koerkamp, 2024a).

2.3 Diagonal transition

Diagonal transition keeps, for each cost s and diagonal, the furthest cell an alignment of cost s reaches, extended along matches at no cost, in $O(ns)$ time in the worst case and $O(n + s^2)$ expected time on random sequences (Ukkonen, 1985; Myers, 1986); WFA generalises it to gap-affine costs, where the wavefront, or front, of each cost has three layers (Marco-Sola *et al.*, 2021), and BiWFA runs it from both ends (Marco-Sola *et al.*, 2023).

2.4 The seed heuristic

A*PA divides the reference into disjoint seeds of fixed length and finds their matches in the query, exactly or with one edit (Groot Koerkamp and Ivanov, 2024). The potential of a seed is the cost r of crossing it unmatched, 1 for exact seeds and 2 for seeds matched with one edit, and a match crosses a seed for less than r. The gap-chaining seed heuristic charges a chain of matches from a cell the cost of each match and, for each gap between consecutive matches and from the last to the end, the larger of the potential of the seeds that start in the gap and the number of diagonals it shifts. Its value, the least charge of any chain, never exceeds the remaining cost, and A*PA keeps the match starts in layers by the value of the best chain each begins (Groot Koerkamp and Ivanov, 2024). A*PA2 prunes matches before the alignment, in place of A*PA's pruning during the search: a match ν with cost $c(\nu)$ and end cell $\mathrm{end}(\nu)$ is kept only if a path from $\mathrm{end}(\nu)$, starting at cost $c(\nu)$, crosses the next $p = 14$ seeds, counting that of ν, while its cost stays below the potential of the seeds it has crossed (Groot Koerkamp, 2024a). A diagonal transition from $\mathrm{end}(\nu)$ tests this, dropping each furthest-reaching point whose cost reaches that potential; a further point on a diagonal has crossed at least as much potential, so no such path is missed. A*PA2 uses the heuristic to drop the rows of each band whose score plus the heuristic exceeds the bound (Groot Koerkamp, 2024a).

3 Methods

3.1 Overview

Each problem is assigned to an algorithm (Table 1; Fig. 1). Unit costs follow A*PA2 (Groot Koerkamp, 2024a): band doubling over 256-column

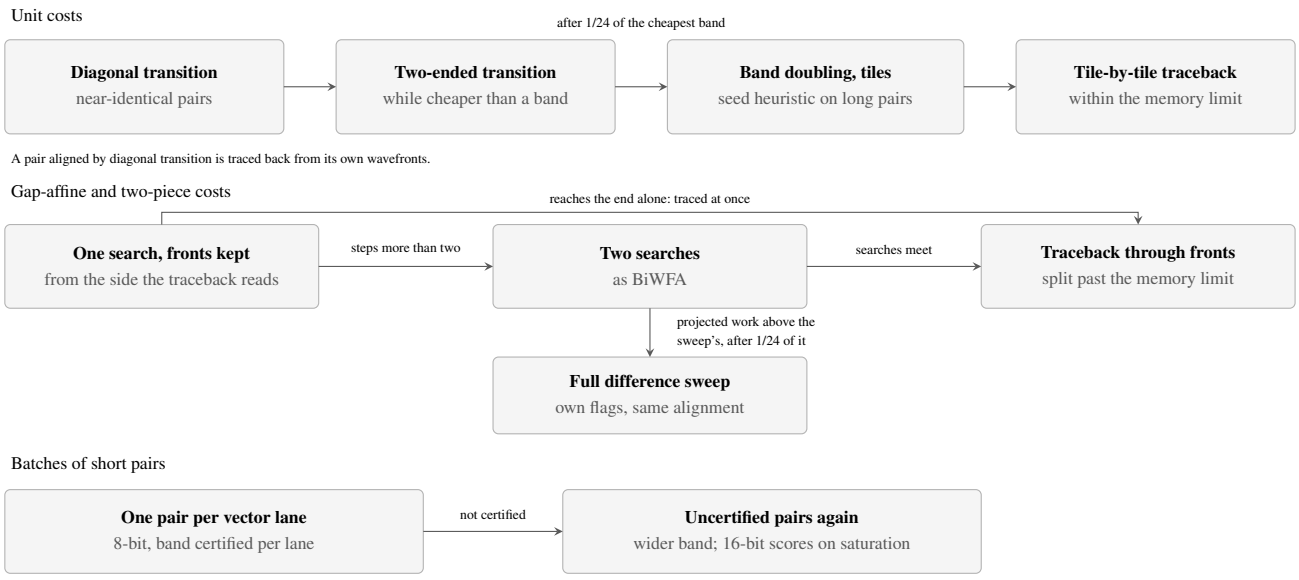


Fig. 1. How dinara-align assigns a pair to an algorithm, and the rule behind each hand-over. At unit costs, diagonal transition from one end and then from both aligns near-identical pairs, handing over to band doubling over bit-parallel tiles only once its projected work exceeds the band's and it has spent one twenty-fourth of the cheapest band (Section 3.10); the tiles are traced back within the memory limit. At gap-affine costs one search grows alone, its fronts kept, and is traced back at once if it reaches the end; a second search joins once one would step more diagonals than two would need (Section 3.9), and a global pair projected to take more work than a full sweep in difference recurrences is handed to that sweep after one twenty-fourth of its work, the alignment the same either way. Batches of short pairs are aligned one pair per vector lane within a band whose optimality each lane certifies (Section 3.6). The figure shows alignments; a unit-cost distance skips the one-ended search.

tiles of bit-parallel words, pruned on long pairs by the gap-chaining seed heuristic (Groot Koerkamp and Ivanov, 2024), with near-identical pairs aligned by diagonal transition alone. Gap-affine costs use the wavefront of WFA and BiWFA (Marco-Sola *et al.*, 2021, 2023). At unit costs, diagonal transition runs first and hands over once its projected remaining work exceeds the alternative's and it has spent a twenty-fourth of it (Section 3.10); its projection, the length and the cost model then choose the next algorithm and its parameters.

At unit, gap-affine and two-piece costs, every engine selects the same alignment among equally good ones, by the backtrace order of WFA2-lib: from the end, an edit into a cell is preferred to a match whenever one is optimal, a substitution before a deletion (a letter of A alone) before an insertion (a letter of B alone), the second gap piece before the first, and a gap extended before it is opened. This rule gives WFA2-lib's CIGARs exactly; the default applies it to both sequences reversed, which places every edit as far left as possible, as variant callers normalise indels. The wavefronts and the tile traceback (Section 3.4) take the furthest-reaching source, which makes the same choice, and the batches and the difference sweep store per-cell flags under the rule. Two exceptions remain: a gap-affine pair whose wavefronts would exceed the memory limit is split and follows the rule within each piece, and alignments under a substitution matrix follow the traceback order of Gotoh's recurrence.

3.2 A shorter critical path for Myers' recurrence

As in Edlib and A*PA2 (Šošić and Šikić, 2017; Groot Koerkamp, 2024a), the query is packed 64 bases per word, and Myers' recurrence (Myers, 1999) advances a word by one column. With match mask E, vertical difference masks P_v, M_v and the incoming negative horizontal difference folded into $E' = E \vee M_h^{\text{in}}$, A*PA2 computes

$$\begin{aligned} c &= (E' \wedge P_v) + P_v, \\ X_h &= (c \oplus P_v) \vee E', \\ P_h &= M_v \vee \neg(X_h \vee P_v), \\ M_h &= P_v \wedge X_h. \end{aligned} \tag{2}$$

The horizontal masks P_h and M_h determine the vertical masks of the next column, so the path from c to them lies on the loop-carried chain. Since $(c \oplus P_v) \vee P_v = c \vee P_v$ and $P_v \wedge (c \oplus P_v) = P_v \wedge \neg c$, the same masks are obtained as

$$\begin{aligned} P_h &= M_v \vee \neg\big(c \vee (E' \vee P_v)\big), \\ M_h &= (P_v \wedge \neg c) \vee (P_v \wedge E'), \end{aligned} \tag{3}$$

where $E' \vee P_v$ and $P_v \wedge E'$ do not depend on c and are computed off the critical path. The vertical update is regrouped in the same way, its negated part computed from E, M_v and the incoming horizontal difference before P_h is known. Together the rewrites shorten the loop-carried chain of a single word from 14 to 8 cycles per column. Myers (1999), Edlib and A*PA2 use the original form, and we are not aware of a previous description of the regrouped one.

Words occupy the lanes of a vector, staggered by one column, so that the horizontal difference leaving one word enters the next through a lane rotation (Wozniak, 1997; Groot Koerkamp, 2024a), and each column waits for the rotation of the previous one. A*PA2 overlaps these waits with two four-lane vectors; on AVX-512, where a vector holds eight words, dinara-align interleaves two eight-word groups, the lower ten columns behind the upper, so that each fills the latency of the other. Bands of two or three words run in general-purpose registers, whose operations have lower latency.

3.3 Band bounds from agreeing estimates

Band doubling is exact for any initial bound, but its running time depends on how close that bound is to the distance. A computation that has reached

Table 1. Algorithms in dinara-align and the changes introduced in this work.

Problem	Algorithm	Changes (section)
Unit costs, divergent pairs	Band doubling over bit-parallel tiles with the gap-chaining seed heuristic (A*PA2)	Regrouped recurrence, interleaved groups (3.2); agreeing estimates (3.3); direct neighbours (3.5); traceback within a memory limit (3.7)
Unit costs, near-identical pairs	Diagonal transition (WFA, BiWFA)	Gathered extension (3.4); bounded-waste hand-over (3.10); per-pair overhead (3.11)
Gap-affine, two-piece	Wavefront, one search or two (WFA, BiWFA); full sweep in difference recurrences for divergent pairs (Suzuki and Kasahara, 2018)	Gathered extension (3.4); one search or two (3.9); bounded-waste hand-over (3.10)
Substitution matrices	Anti-diagonal vector sweep (Wozniak, 1997)	Score-certified band, cache-sized strips (3.8)
Local alignment	End found by a forward sweep (Zhao et al., 2013)	Backward wavefront from the best end (3.8)
Batches of short pairs	Inter-sequence vectorisation (Rognes, 2011; Rahn et al., 2018; Daily, 2016)	Certified per-lane bands (3.6)

cost s at anti-diagonal ρ estimates the distance as $\hat{D} = s(n+m)/\rho$, which is reliable when edits are spread evenly and too high by orders of magnitude when they gather, as at the ends of nanopore reads. dinara-align therefore takes two estimates, $\hat{D}_1$ from a one-ended diagonal-transition search stopped after eight edits and $\hat{D}_2$ from the two-ended search where it handed over. Without seeds that chain well, the first bound is aimed just above $\hat{D}_1$ when $\max(\hat{D}_1, \hat{D}_2) \leq \frac{4}{3}\min(\hat{D}_1, \hat{D}_2)$, or when the search stopped within 16 edits; otherwise, as in A*PA2, it starts at most 256 above the known lower bound and each retry at most doubles it. A failed round aims the next bound from where it stopped, and the first round estimates the distance again after one eighth, one quarter and one half of the columns, lowering its bound or abandoning the round. A distance is accepted only within the bound its round ended on, and these rules change only the sequence of bounds, so the result stays exact.

Within a round, the band drops the rows at the top and bottom of each tile whose score plus the heuristic exceeds the bound, as A*PA2 does. A score changes by at most one per row, and the heuristic by at most its climb γ (one without seeds or with exact seeds, two with inexact ones), so a row that exceeds the bound by y rules out itself and the next $\lceil y/(1+\gamma) \rceil - 1$ rows, which are skipped unread. With seeds, when at least one seed in 32 chains from the origin, the first bound starts 64 above the heuristic at the origin, which on long reads lies within a few per cent of the distance.

3.4 Diagonal transition for near-identical pairs

Diagonal transition extends each diagonal of a wavefront along its run of matches. dinara-align compares eight letters at a time by the exclusive-or of two 64-bit words and a count of trailing zeros, and on AVX-512 extends eight diagonals together, their letters loaded by gather instructions under branch-free masks (Supplementary Section S1.1). The same vectorised step serves three uses: near-identical pairs, from one or both ends (Marco-Sola *et al.*, 2023); the traceback inside each tile, restricted to the diagonals that can still reach the traced cell; and the three-layer gap-affine front.

A pair whose diagonal transition finishes within its budget is aligned from its wavefronts alone. From both ends, as in BiWFA, the forward wavefronts are grown on to the proven distance, each cost kept only on the diagonals on which the backward wavefronts show that an optimal path passes. The backtrace takes the furthest-reaching source at each cost (Section 3.1) and writes the CIGAR run by run.

3.5 Seed heuristic: direct neighbour construction

The reference is divided into disjoint seeds of 12 bases, matched exactly, or of 16 bases, matched with at most one edit (Groot Koerkamp and Ivanov, 2024); a 16-base seed matched with one edit matches one of its 8-base halves exactly, so each half indexes a table. The lookups for 16 query rows are issued together, so that their cache misses overlap, and only seeds whose matches could still chain to the end of the pair are considered.

An exact match μ of a 16-base seed at query rows $[j, j+16)$ has four neighbouring matches with one edit, fewer at the ends of the query: two share its start, at rows $[j, j+15)$ and $[j, j+17)$, and two share its end, at $[j+1, j+16)$ and $[j-1, j+16)$. Because chaining is all-or-nothing, a neighbour one diagonal away may chain where μ cannot, and the neighbours are required for the heuristic to remain a lower bound. Rather than locating them by search and testing each for local pruning, dinara-align inserts them whenever μ is retained, which is justified by the following lemma, in which the search is the diagonal transition of local pruning (Section 2.4).

Lemma 1. If the search from an exact match μ finds no path across the next p seeds, it finds none from any of its four neighbours.

Proof idea. The search from μ dominates that from each neighbour: at every cost and on every diagonal it reaches at least as far, and a further point has crossed at least as much potential, so it is dropped only if the nearer one is too (proof in Supplementary Section S6).

Every neighbour that the search would keep is therefore inserted. As in A*PA2, the implementation also keeps a match whose search runs into the start of a match already kept on the same diagonal, which keeps more matches and drops none. A match is thus dropped only if its search would find no path: either the search ran and found none, or the match neighbours an exact match whose search found none (Lemma 1). Dropping such matches keeps the heuristic a lower bound (Lemma S1), and each further match kept can only lower it, so the alignment stays exact. Each neighbour is placed in its layer from its own end's chain, without a search of its own.

3.6 Certified bands for inter-sequence batches

Batches of short pairs are aligned with one pair per vector lane (Rognes, 2011; Rahn *et al.*, 2018) in saturating 8-bit lanes (Zhao *et al.*, 2013; Daily, 2016); a score that saturates is computed again in 16-bit lanes, and an alignment that saturates goes to the single-pair engines. Rather than computing each matrix in full, the lanes of a group compute the band of diagonals $[d_\ell, d_h]$ containing the main diagonal 0 and each pair's end diagonal δ, with one further diagonal on either side; pairs are grouped by end diagonal, so the band stays narrow. Its optimality is certified by the following bound, a cost form of the out-of-band bounds of Gibrat (2018) and Fujiki *et al.* (2020) for gap-affine and two-piece costs, with a strict form for tie-breaking.

Proposition 1. Let C be the least cost of an alignment within $[d_\ell, d_h]$, and let $L(d) = g(|d|) + g(|d-\delta|)$, or ∞ for a diagonal outside the

matrix. If $C \le \min\big(L(d_h+1), L(d_\ell-1)\big)$, then C is the optimal cost; if the inequality is strict, every optimal alignment lies within the band.

Proof idea. An alignment that leaves the band must open a gap reaching the diagonal beyond it and another returning to the end diagonal, which together cost at least L of that diagonal (proof in Supplementary Section S6).

Each vector lane of an inter-sequence batch evaluates its own certificate. When every optimal alignment lies in the band, each in-band score on an optimal path is exact, so the tie rule makes the same choices as in the full matrix, and the strict form guarantees that the alignment it selects lies within the band. Pairs that fail are recomputed over every diagonal that a path at most as cheap could visit, grouped by power-of-two classes of that band's width so that the lanes of a group finish together.

3.7 Traceback within a memory limit

The traceback of a band traces each tile from the boundary recorded on its left. On 1 Mbp pairs at 20% errors, keeping every boundary made an alignment grow peak memory by up to 175 MiB, so the boundaries are held within the alignment's memory limit, 80 MiB by default: beyond it only one boundary in two is kept, then one in four, and so on. Each tile also records the state the band carries into it, from which the traceback recomputes a stretch of tiles starting at the last kept boundary (Supplementary Section S1.1). Every step of a band depends only on this state and on the scores it computes, so each boundary is recovered as the round left it and the alignment is the same byte for byte; splitting the pair instead, as QuickEd does (Doblas *et al.*, 2025), resolves ties within each part. On the 1 Mbp pairs at 20% errors, the growth of peak memory fell from 175 to 133 MiB on the worst pair when this was introduced, for 9% more time (Section 4.2 gives the reported commit's).

3.8 Gap-affine and score-based alignment

A scoring scheme with match score α, mismatch score β and gap scores $-(o+ke)$ can be converted into costs for global alignment (Eizenga and Paten, 2022). Every alignment covers $n+m$ letters, so its score S and a gap-affine cost C satisfy

$$\begin{aligned} S &= \tfrac{1}{2}\big(\alpha(n+m) - C\big), \\ C &= 2(\alpha-\beta)\,X + \sum_{\text{gaps}} \big(2o + k(2e+\alpha)\big), \end{aligned} \tag{4}$$

where X is the number of mismatches, so the wavefront computes score-maximising alignments. Local alignment locates the end of the optimal alignment with a forward sweep, as in SSW (Zhao *et al.*, 2013), and extends a wavefront back from it to the first cell attaining the maximal score, which avoids the correction pass that striped layouts may need (Farrar, 2007).

The wavefront keeps five bytes per diagonal for the traceback, and a pair whose fronts would exceed the memory limit is split where an optimal path crosses, as in BiWFA, following the tie rule within each piece (Section 3.1). A global pair projected to take more work than a full sweep in 8-bit difference recurrences (Suzuki and Kasahara, 2018) is handed to that sweep (Section 3.10), which keeps the predecessor flags of Section 3.1; unrelated 5 kbp pairs then aligned in 4.8 ms instead of 32.9 ms.

Under a substitution matrix, anti-diagonals (Wozniak, 1997) are computed in 16-bit lanes while the scores provably fit, in strips of rows that stay in the first-level cache, and a global alignment is traced in a band of diagonals that grows until its own score proves that every optimal path lies inside it (Supplementary Section S1.4).

3.9 One search or two at gap-affine costs

WFA grows the wavefront from one end of the pair and keeps every front for the traceback (Marco-Sola *et al.*, 2021); BiWFA grows it from both ends and keeps only the latest fronts (Marco-Sola *et al.*, 2023). Two searches must also prove where an optimal path crosses between them, so they stop only once $2\min(s_1, s_2) \ge C^* - 1 + \theta$, where s_1, s_2 are their costs, C^* the cheapest meeting found and $\theta = o + w$ a margin: o is the largest gap-open cost and $w = \max(x, o+e)$ the largest cost of a single edit, over both pieces for two-piece costs. Every optimal path has a cell whose costs from the two ends differ by at most w, and a meeting inside a gap pays its opening once for both halves; BiWFA stops by the same margin. A score-only computation, which needs no meeting cell, stops sooner: once its best is at most one more than the highest cost to which either search has checked the other's start. With $W(s)$ the number of diagonals a search steps to reach cost s, two searches need about $2W((C_{\text{opt}}+\theta)/2)$ steps for a pair of cost C_{opt}, plus the setup of the second, and one search $W(C_{\text{opt}})$. Where the fronts widen with the cost, $W(s)$ grows with s^2 and one search is cheaper while $C_{\text{opt}} < (1+\sqrt{2})\,\theta$; where the ends of the sequences bound the fronts, it is always cheaper. Most 250 bp Illumina reads cost less than θ (14 at (4, 6, 2)), and on them two searches took 2.82 µs per read in the benchmark runner, against 1.55 µs for WFA.

dinara-align therefore starts with a single search, the one whose fronts the traceback reads (the backward search under the default tie rule), keeps its fronts and traces the alignment as soon as the search reaches the far corner. Every cost s passed without reaching it shows that $C_{\text{opt}} > s$, and the search measures W as it grows: it continues alone while $s < \theta$, costs the second search would pass in any case, and then while $W(s) \le 2W(\lfloor (s+\theta)/2 \rfloor)$, that is, while it has stepped no more diagonals than two searches would need for the cheapest cost still possible. It also hands over at half the memory limit or when a band leaves it no reachable cell. The rule has no tuned constant: it hands over at about 2.4θ where the fronts widen with the cost, and never where the sequences bound the fronts from early costs on, where it aligns exactly as WFA does.

Proposition 2. Suppose the first search stops at cost σ without reaching the far corner, and the second search, growing from cost 0, checks each new front against the last $w+1$ fronts of the first, costs $\sigma - w$ to σ, while its own cost is at most σ, after which the two alternate as in BiWFA, each new front checked against the other side's last $w+1$. When they stop, C^* is the optimal cost and its meeting lies on an optimal path.

Proof idea. On every optimal path the costs from the two starts change by at most w a move, so some cell has them within w of each other, and the two phases check every such cell before the stopping rule can end the searches (proof in Supplementary Section S6).

An alignment traced from the unpruned fronts of the single search is the one the two searches would have traced: the traceback steps only from a cell to a predecessor on an optimal path, and the fronts that two searches prune hold no such cell, so the tie rule is unchanged. Starting with one search took a 250 bp Illumina read at (4, 6, 2), through one aligner, from 2.41 to 1.52 µs, and the measured-work rule, in place of a single search that grew alone only through its first w costs, saved a further 5%, a third on the pairs costing 9 to 32; nanopore and HiFi reads were unchanged.

3.10 Handing over on a projection, with bounded waste

Two hand-overs rest on projections: diagonal transition hands over to band doubling once its remaining work is projected to exceed a band's, and the gap-affine wavefront to the difference sweep once its projected total exceeds the sweep's. A projection from the edits seen so far is reliable when edits are spread evenly, but can exceed the distance a hundredfold when they gather at the ends, as on nanopore reads with adapters or poly-A tails, and so hand a pair that the search would finish in microseconds to a band over the whole matrix. dinara-align hands over on a projection

only once the work already spent has reached one twenty-fourth of the cheapest alternative that could still be needed: for diagonal transition, the band whose bound equals the cost already reached, than which no band is cheaper since the distance is at least that cost; for the wavefront, the full sweep. Once the projection favours the alternative, the rule thus delays the hand-over by at most about 4% (1/24, in the units of the cost model) of the work the alternative needs in any case, while a search that would have finished is no longer abandoned on a projection from a handful of edits. At gap-affine costs, the rule took the ont-10k reads from 2.33 to 1.56 ms and the ont-50k reads from 30.5 to 21.0 ms (Supplementary Table S1). For an alignment, the one-ended diagonal transition also waits until the projected work exceeds the band's by a factor $1 + 1.5/\sqrt{q}$ after q edits, 1.5 times the relative error of a projection from q edits.

3.11 Per-pair overhead on short reads

A 250 bp read aligns in well under a microsecond, so work that does not depend on the pair dominates. An aligner object keeps every buffer its searches use from pair to pair, one aligner per thread, as each competitor's benchmark runner keeps its own; both sequences are encoded, and reversed for the leftmost tie rule, in one vectorised pass; and results pass between internal calls as plain values rather than as optionals, which the compiler copied a byte at a time. Together these took a unit-cost alignment of a 250 bp Illumina read, timed in-process through one aligner, from 0.97 to about 0.40 µs (Supplementary Table S1); the benchmark runner, which reads and checks each pair, adds to this (Section 4).

3.12 Thresholds measured against the code

The choice of algorithm and the bounds of each search rest on 41 thresholds, each set from measurements on the code of its time. We built dinara-align with each halved and with each doubled and timed the 82 builds against the unchanged one (Supplementary Section S1.1): about thirty moved no sample by more than 3%, and most others traded one dataset against another. One had become outdated, the length from which seeds match with one edit, which had cost 100 kbp pairs at 10% errors a third of their running time; examining the slowest results found two more (Supplementary Section S1.1).

3.13 Validation

A differential fuzzer checks every mode, cost model, band, cost limit and memory limit on random pairs against a full-matrix implementation that shares no code with the library, for validity and optimality, and checks the tie rule: an alignment under a cost limit against the same pair without one, and the default rule against WFA2-lib's over both sequences reversed. A unit test shows that a unit-cost alignment is the same byte for byte under any memory limit, and all 305 pairs of WFA2-lib's regression set give its scores in every mode it tests and its CIGARs under its tie rule (Supplementary Section S1.7).

4 Results

4.1 Experimental setup

All measurements were taken on an Intel Core i9-7900X (Skylake-X, AVX-512), its ten cores at a fixed 3.3 GHz with turbo boost and hyper-threading disabled, with dinara-align compiled by Mojo 1.1.0 (Modular Inc., 2026) at commit `036e12b` (gap-affine costs at `a6d9166`; the other modes and the batches at `289bcf0`, before the one-search start of Section 3.9), and every tool compiled for the native instruction set except BSAlign's gap-affine kernel, which its own build keeps to SSE4.2. We combined the protocols of A*PA2 (Groot Koerkamp, 2024a) and QuickEd (Doblas *et al.*, 2025). The real datasets come from A*PA2 and QuickEd. From A*PA2's release (Groot Koerkamp, 2024b) we use the nanopore sets ont-1k, ont-10k, ont-50k and ont-500k, named by length class (mean 0.83, 3.6, 9.6 and 642 kbp), CHM13 reads paired with the CHM13 reference; ont-500k-genvar, ultra-long reads of another individual against the same reference, which adds genetic variation and gaps of up to 42 kbp; and 30 kbp SARS-CoV-2 genomes. We call the two 500k sets ultra-long. For QuickEd's Illumina 2×250 bp and PacBio HiFi technologies we use HG002 reads from Genome in a Bottle (Zook *et al.*, 2016) in one region of chromosome 20, picked with mapping quality at least 20 on GRCh38, each mapped by minimap2 (Li, 2018) to both haplotypes of HG002's own assembly (Hansen *et al.*, 2026) and paired with the stretch it covers, its soft-clipped ends trimmed, so that pairs differ mainly by sequencing errors. The simulated datasets are QuickEd's random sequences (Doblas *et al.*, 2024) of 10 kbp to 1 Mbp whose pairs differ by single-base substitutions, insertions and deletions spread uniformly at nominal error rates of 1, 5, 10 and 20%, modelling no sequencing technology; the 20% sets measure 17% divergence (Supplementary Table S6). Each dataset was sampled to about 2 Mbp and at least four pairs. Each tool aligned each pair once, with traceback, on one pinned core, through one aligner object per sample as its own benchmark runner does, with a 30 s budget per sample. The other tools' times were measured once at their pinned commits and kept; dinara-align was timed again at the commits above. Timings quoted in Section 3 were recorded when each change was made, against the version before it (Supplementary Table S1). The exact aligners compared are A*PA2-full (with the seed heuristic and pruning) and A*PA2-simple (band doubling alone), A*PA (Groot Koerkamp and Ivanov, 2024), Edlib (Šošić and Šikić, 2017), WFA and BiWFA, which denote WFA2-lib in its high-memory and bidirectional modes (Marco-Sola *et al.*, 2021, 2023), QuickEd, BSAlign (Shao and Ruan, 2024), KSW2 (Li, 2018), parasail (Daily, 2016) and SSW (Zhao *et al.*, 2013), each on the workloads it supports. On the pairs any two tools completed, their costs agreed, and at unit costs they agreed with the published A*PA2 results on its datasets. The other alignment modes were timed in-process on simulated workloads, the faster of two passes (Supplementary Table S3), and the batches on 500,000 simulated pairs of 148 bp at 1% error (Supplementary Table S8).

4.2 Speed and memory

At unit costs dinara-align had the lowest mean time per alignment of the exact aligners on every dataset. On the real data (Table 2) it was 1.4 to 2.4 times faster than the fastest alternative on the nanopore reads and 2.5 times on the SARS-CoV-2 genomes, and 1.95 and 5.2 times faster than WFA, the fastest of the others there, on the Illumina and HiFi reads. On the simulated data (Fig. 2; Supplementary Table S5) it was 1.1 to 2.5 times faster than the fastest alternative: 100 kbp pairs at 1, 5, 10 and 20% errors took 1.5, 5.1, 7.3 and 15.7 ms, against 7.5, 8.4, 13.9 and 36.7 ms for A*PA2-full and 12.5, 24.0, 39.0 and 62.8 ms for QuickEd. Its narrowest margin was on 10 kbp pairs at 5%, 0.267 against 0.288 ms for A*PA2-simple, which lies within the variation between builds (Section 5). Over all 24 datasets QuickEd took 3.0 to 75 times as long, 8.0 times in the geometric mean.

At gap-affine costs dinara-align was the fastest on every dataset that any tool completed (Table 3; Table 4), 1.07 to 3.3 times faster than the faster of WFA and BiWFA, with its narrowest margin on the Illumina reads, 1.45 against 1.55 µs for WFA. It completed more of the simulated pairs of 100 kbp or more than any other tool, 47 against 41 for BiWFA, but it completed no ont-500k read and one of the four ont-500k-genvar reads within the budget. In the other modes it was 1.0 to 22 times faster than the fastest tool offering the same mode (Supplementary Table S3), two of the margins lying within the variation between builds. Global scores of 500,000 short-read pairs took 230 ms on one core and 27 ms on ten at (1, 2, 1), 4.6 and 4.3 times faster than WFA2-lib (Supplementary Table S8).

Table 2. Unit-cost alignment of the real datasets by every exact aligner: mean time per alignment in ms, traceback included, and below it the peak resident memory of the run in MiB, as QuickEd reports it. t.o. marks a tool that completed no pair within 30 s, o.o.m. one the 8 GiB limit stopped, and crash one that crashed (BSAlign's edit distance on every pair of 500 kbp or more, with or without the limit). The simulated datasets are in Fig. 2 and Supplementary Table S5. Bold marks the lowest mean of the tools that completed the most pairs.

Dataset	dinara-align	A*PA2-full	A*PA2-simple	A*PA	QuickEd	Edlib	BiWFA	WFA	BSAlign	KSW2
Mean time per alignment										
ont-1k (830 bp)	**0.0211**	0.0806	0.0480	0.612	0.0748	0.124	0.0443	0.0311	0.119	0.539
ont-10k (3.6 kbp)	**0.156**	0.391	0.247	15.3	0.688	1.09	0.741	0.599	1.30	13.1
ont-50k (9.6 kbp)	**0.580**	1.27	0.944	263	2.31	6.14	7.43	6.38	12.3	167
ont-500k (642 kbp)	**118**	283	687	t.o.	847	5,000	6,330	o.o.m.	crash	o.o.m.
ont-500k-genvar (663 kbp)	**156**	225	511	t.o.	1,390	5,230	6,200	o.o.m.	crash	o.o.m.
SARS-CoV-2 (30 kbp)	**0.286**	2.00	0.728	6.49	3.90	8.18	0.909	0.987	47.1	68.5
Illumina (248 bp)	**0.000576**	0.0145	0.00410	0.0137	0.0165	0.0233	0.00204	0.00112	0.0206	0.0124
PacBio HiFi (14 kbp)	**0.0126**	0.977	0.259	1.05	0.950	3.35	0.0861	0.0659	12.2	3.09
Peak resident memory (MiB)										
ont-1k	14	5	5	6	2	7	7	8	2	7
ont-10k	16	6	6	39	6	8	8	39	25	72
ont-50k	19	8	7	858	18	8	10	187	423	650
ont-500k	64	89	90	t.o.	26	15	29	o.o.m.	crash	o.o.m.
ont-500k-genvar	64	57	55	t.o.	27	16	27	o.o.m.	crash	o.o.m.
SARS-CoV-2	18	7	6	19	15	8	9	26	215	151
Illumina	13	5	5	5	2	7	7	7	2	7
PacBio HiFi	13	7	6	6	4	8	8	9	127	21

Table 3. Gap-affine alignment of the real datasets at costs (4, 6, 2) by every exact aligner that takes them: mean time per alignment in ms, a 30 s budget per sample. Brackets give the pairs completed where not all were; t.o. marks a tool that completed none and o.o.m. one the 8 GiB limit stopped. Every dataset is in Supplementary Table S4. Bold marks the lowest mean of the tools that completed the most pairs.

Dataset	dinara-align	WFA	BiWFA	BSAlign	KSW2
ont-1k	**0.105**	0.188	0.308	0.336	1.10
ont-10k	**1.60**	4.89	5.86	7.84	30.0
ont-50k	**21.5**	63.4	54.9	87.0	341 [81/104]
ont-500k	t.o.	o.o.m.	t.o.	o.o.m.	o.o.m.
ont-500k-genvar	22,000 [1/4]	o.o.m.	t.o.	o.o.m.	o.o.m.
SARS-CoV-2	**0.710**	2.05	1.78	341	87.9
Illumina	**0.00145**	0.00155	0.00281	0.0539	0.0128
PacBio HiFi	**0.0950**	0.227	0.231	86.0	9.72

An alignment of an ultra-long read increased peak memory by at most 48 MiB, against 82 MiB for A*PA2-full, and of a 1 Mbp pair at 20% errors by at most 123 MiB, against 184 MiB (Supplementary Table S7). This growth includes the seed heuristic and the band; the stored tile boundaries stayed within the default limit of 80 MiB (Section 3.7). QuickEd stayed near 25 MiB. On short reads dinara-align's peak was higher than the others' (Table 2): the Mojo runtime and the sample, read whole, take about 13 MiB.

5 Discussion

dinara-align was the fastest exact aligner on every dataset at unit costs, and on every dataset that any tool completed at gap-affine costs. This is consistent with choosing an algorithm for each pair: no single engine was fastest across these workloads, and since every engine is exact, choosing never forfeits the guarantee or changes the alignment. Three observations stand out. First, each technique paid where its engine does the work (Supplementary Table S2): on the ultra-long reads, switching off direct neighbour construction, which reduces work outside the inner loop, slowed alignment by 9 and 16%, more than either change to the loop itself (at most 5%), while the interleaved groups paid most on the widest bands and gathered extension where diagonal transition does most of the work. Second, on short reads the work that does not depend on the pair dominates: a 250 bp alignment at gap-affine costs once took 5.2 µs through a fresh call and 2.4 µs through a reused aligner (Supplementary Table S1), so a benchmark should call every tool as its own runner does. Third, thresholds that encode a cost model become outdated as the code changes, and measuring them again belongs in the evaluation of any aligner that selects its algorithm by thresholds. Gap-affine alignment of ultra-long reads remains open: no tool completed an ont-500k read within 30 s. Where memory matters more than time, Edlib, QuickEd and BiWFA (15 to 29 MiB on ultra-long reads) use less than dinara-align's 64 MiB. The measured-work rule, the regrouped recurrence and the per-lane certificate do not depend on this implementation and apply to other wavefront, bit-parallel and inter-sequence aligners.

5.1 Limitations

1. The evaluation covers one x86 processor with AVX-512, and single-thread latency except for the batches; the AVX2 and NEON configurations are not evaluated here.
2. The evaluated samples also guided development: several changes were motivated and checked on the Illumina, HiFi and ont-500k-genvar samples, and some thresholds, in particular the AVX-512 gates for inexact seeds, were set on uniform 100 kbp pairs like those of Fig. 2a, so results on unseen data may be less favourable.

Table 4. Gap-affine alignment of QuickEd's simulated datasets, random sequences with uniformly spread substitutions, insertions and deletions, at costs (4, 6, 2) by every exact aligner that takes them: mean time per alignment in ms, a 30 s budget per sample. Brackets give the pairs completed where not all were; t.o. marks a tool that completed none and o.o.m. one the 8 GiB limit stopped. Datasets no tool completed are left out; every dataset is in Supplementary Table S4. Bold marks the lowest mean of the tools that completed the most pairs.

Dataset	dinara-align	WFA	BiWFA	BSAlign	KSW2
10 kbp, 1%	**0.199**	0.416	0.646	37.0	21.3
10 kbp, 5%	**2.32**	7.14	8.72	37.0	96.1
10 kbp, 10%	**7.06**	23.1	27.1	37.3	202
10 kbp, 20%	**20.1**	66.0	75.1	37.4	375 [81/99]
100 kbp, 1%	**11.9**	39.9	39.0	o.o.m.	2,420
100 kbp, 5%	**331**	808	714	o.o.m.	9,170 [3/9]
100 kbp, 10%	**1,160**	o.o.m.	2,430	o.o.m.	17,300 [1/10]
100 kbp, 20%	**3,610 [8/10]**	o.o.m.	7,360 [4/10]	o.o.m.	o.o.m.
500 kbp, 1%	**477**	1,290	942	o.o.m.	o.o.m.
500 kbp, 5%	**9,910 [3/4]**	o.o.m.	19,400 [1/4]	o.o.m.	o.o.m.
1 Mbp, 1%	**2,300**	o.o.m.	3,940	o.o.m.	o.o.m.

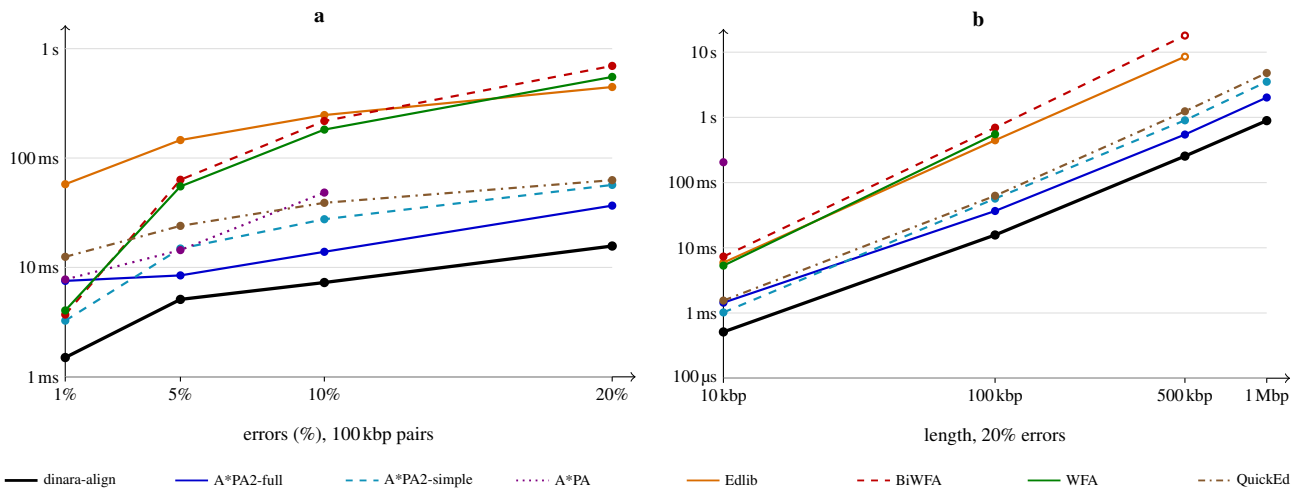


Fig. 2. Unit-cost alignment of QuickEd's simulated pairs, mean time per alignment on a logarithmic scale. (a) 100 kbp pairs by error rate. (b) Pairs at 20% errors by length; lines end where a tool completed no pair within 30 s or the 8 GiB limit stopped it, and hollow points mark tools that completed only some pairs. BSAlign and KSW2 are left out for legibility; their times are in the repository's benchmark tables. dinara-align had the lowest mean time at every length and error rate.

3. As in the A*PA2 protocol, each pair was aligned once, and the ultra-long and the largest simulated sets were sampled to four pairs. The other tools' times were kept from earlier runs of the same pinned builds rather than measured in the same session. On the Skylake-X some running times varied by up to a third between builds that did not change the code involved; the ablation controls for this (Supplementary Table S2), but margins of less than about 10% in the comparison with other tools lie within it.
4. The other alignment modes and the batches were measured at an earlier commit than the unit-cost and gap-affine results (Section 4.1).
5. The Illumina and HiFi reads come from one region of chromosome 20 and exclude reads that minimap2 leaves more than a tenth clipped, such as those spanning structural variants.
6. The ablation switches off one technique at a time, so it does not separate their interactions.

5.2 Future work

1. a cheap upper bound on the distance: with the true distance supplied by an oracle, starting band doubling at it saves up to 27% on 100 kbp pairs at 20% errors, but starting 15% above it loses up to 23% on ultra-long reads, whereas capping the bound at an upper bound within 5% of the distance saves up to 27% on 1 Mbp pairs at 20% (Supplementary Section S5);
2. reusing computation across band-doubling rounds, as in the incremental doubling of A*PA2, once its exactness for these tiles is proven;
3. faster construction of the seed heuristic, whose scalar lookups dominate on long divergent pairs, and a seed heuristic for gap-affine costs, at which no tool completed a 1 Mbp pair beyond 1% errors;
4. measurements on further processors, and an evaluation of the GPU kernels the library provides for batches (Supplementary Section S1.5).

Author contributions

Konstantinos Kyriakidis (Conceptualization [lead], Methodology [lead], Software [lead], Validation [lead], Formal analysis [lead], Investigation [lead], Data curation [lead], Visualization [lead], Writing—original draft [lead]), Benedict Paten (Supervision [lead], Writing—review & editing [equal]).

Supplementary data

Supplementary data are available at *Bioinformatics* online.

Conflict of interest

None declared.

Funding

This work received no specific funding.

Data availability

The software, benchmark harness and the commits of every compared tool are available at `https://github.com/kokyriakidis/dinara-align`. The benchmarks are reproduced by `pixi run bench` (unit and `--affine` costs), `bench-modes`, `bench-batch` and `bench-ablation`. Release builds target the baseline processor of each platform; the AVX-512 code paths require a build for the host processor.

References

Daily, J. (2016). Parasail: SIMD C library for global, semi-global, and local pairwise sequence alignments. *BMC Bioinformatics*, **17**, 81.

Doblas, M., Lostes-Cazorla, O., Aguado-Puig, Q., Iñiguez, C., Moretó, M., and Marco-Sola, S. (2024). QuickEd simulated datasets. Zenodo.

Doblas, M., Lostes-Cazorla, O., Aguado-Puig, Q., Iñiguez, C., Moretó, M., and Marco-Sola, S. (2025). QuickEd: high-performance exact sequence alignment based on bound-and-align. *Bioinformatics*, **41**(3), btaf112.

Eizenga, J. M. and Paten, B. (2022). Improving the time and space complexity of the WFA algorithm and generalizing its scoring. bioRxiv.

Farrar, M. (2007). Striped Smith–Waterman speeds database searches six times over other SIMD implementations. *Bioinformatics*, **23**(2), 156–161.

Fujiki, D., Wu, S., Ozog, N., Goliya, K., Blaauw, D., Narayanasamy, S., and Das, R. (2020). SeedEx: A genome sequencing accelerator for optimal alignments in subminimal space. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 937–950.

Gibrat, J.-F. (2018). A short note on dynamic programming in a band. *BMC Bioinformatics*, **19**, 226.

Gotoh, O. (1982). An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, **162**(3), 705–708.

Groot Koerkamp, R. (2024a). A*PA2: Up to 19× faster exact global alignment. In S. P. Pissis and W.-K. Sung, editors, *24th International Workshop on Algorithms in Bioinformatics (WABI 2024)*, volume 312 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:25. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

Groot Koerkamp, R. (2024b). pa-bench datasets. `https://github.com/pairwise-alignment/pa-bench/releases/tag/datasets`.

Groot Koerkamp, R. and Ivanov, P. (2024). Exact global alignment using A* with chaining seed heuristic and match pruning. *Bioinformatics*, **40**(3), btae032.

Hansen, N. F., Dwarshuis, N., Ji, H. J., Rhie, A., *et al.* (2026). A complete diploid human genome benchmark for personalized genomics. *Cell*, **189**(16), 4857–4875.e31.

Li, H. (2018). Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, **34**(18), 3094–3100.

Liu, D. and Steinegger, M. (2023). Block aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices. *Bioinformatics*, **39**(8), btad487.

Marco-Sola, S., Moure, J. C., Moreto, M., and Espinosa, A. (2021). Fast gap-affine pairwise alignment using the wavefront algorithm. *Bioinformatics*, **37**(4), 456–463.

Marco-Sola, S., Eizenga, J. M., Guarracino, A., Paten, B., Garrison, E., and Moreto, M. (2023). Optimal gap-affine alignment in $O(s)$ space. *Bioinformatics*, **39**(2), btad074.

Modular Inc. (2026). Mojo programming language. `https://docs.modular.com/mojo/`.

Myers, E. W. (1986). An $O(ND)$ difference algorithm and its variations. *Algorithmica*, **1**(1–4), 251–266.

Myers, G. (1999). A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*, **46**(3), 395–415.

Prjibelski, A. D., Korobeynikov, A. I., and Lapidus, A. L. (2019). Sequence analysis. In *Encyclopedia of Bioinformatics and Computational Biology*, volume 3, pages 292–322. Elsevier.

Rahn, R., Budach, S., Costanza, P., Ehrhardt, M., Hancox, J., and Reinert, K. (2018). Generic accelerated sequence alignment in SeqAn using vectorization and multi-threading. *Bioinformatics*, **34**(20), 3437–3445.

Rognes, T. (2011). Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation. *BMC Bioinformatics*, **12**, 221.

Shao, H. and Ruan, J. (2024). BSAlign: a library for nucleotide sequence alignment. *Genomics, Proteomics & Bioinformatics*, **22**(2), qzae025.

Šošić, M. and Šikić, M. (2017). Edlib: a C/C++ library for fast, exact sequence alignment using edit distance. *Bioinformatics*, **33**(9), 1394–1395.

Suzuki, H. and Kasahara, M. (2018). Introducing difference recurrence relations for faster semi-global alignment of long sequences. *BMC Bioinformatics*, **19**(Suppl 1), 45.

Ukkonen, E. (1985). Algorithms for approximate string matching. *Information and Control*, **64**(1–3), 100–118.

Wozniak, A. (1997). Using video-oriented instructions to speed up sequence comparison. *Computer Applications in the Biosciences (CABIOS)*, **13**(2), 145–150.

Zhao, M., Lee, W.-P., Garrison, E. P., and Marth, G. T. (2013). SSW library: An SIMD Smith-Waterman C/C++ library for use in genomic applications. *PLoS ONE*, **8**(12), e82138.

Zook, J. M., Catoe, D., McDaniel, J., *et al.* (2016). Extensive sequencing of seven human genomes to characterize benchmark reference materials. *Scientific Data*, **3**, 160025.